DIGITAL DESIGN REPORT

4-bit ALU with shift-based multiply and divide

Vivado 2026.1 Course Project

Student Name	**Tran Quan Bao**
Roll Number	**SE190570**

Project Overview:

This project implements a 4-bit ALU supporting ten operations; addition (ADD), subtraction (SUB), logic, and bit-shift operations are executed in a single cycle, whereas multiplication (MUL) and division (DIV) utilize a four-step process based on bit-shifting following an initial data-loading cycle.

Part A. Design

The design is partitioned into a top-level module **alu_4bit** and seven dedicated sub-modules to ensure modularity and scalability:

control_unit: Decodes opcode [3:0], handles handshake controls (start, busy, done), and synchronizes multi-cycle operations.

adder_sub_4bit: Performs 4-bit addition and subtraction using a cascade of 4 basic Full Adder cells (**fa_cell**) arranged in a ripple-carry architecture.

logic_unit: Executes bitwise combinational logic functions including AND, OR, XOR, and NOT.

shifter_unit: Implements single-cycle logical shifts (SHL for left shift, SHR for right shift).

muldiv_core: Sequential arithmetic core driven by a finite state machine (FSM) implementing Shift-and-Add multiplication and Restoring division over 4 clock cycles.

result_mux: Multiplexes the computational output from the respective functional unit onto the primary bus result [7:0].

flag_gen: Generates real-time condition flags: Carry (carry), Zero (zero), Negative (negative), Overflow (overflow), and Division-by-Zero (div_by_zero).

Part B. Testbench

The verification framework is implemented as an automated self-checking testbench (**alu_4bit_tb.v**):

Part 1 (Single-Cycle Combinational Suite): Exhaustively tests all possible combinations (16×16) across the 8 combinational opcodes, generating 3584 automated verification checks.

Part 2 (Multi-Cycle Sequential Suite): Exhaustively verifies sequential operations (MUL and DIV) across 512 dynamic corner cases, monitoring timing transitions (start, busy, done).

Part C. Truth Tables and Step-by-Step Calculation Tables

1. Functional ALU Truth Table

Opcode	Op	Function / Expression	Result Width	Cycles	Active Flags
0000	ADD	R = A + B	[3:0]	1	C, V, Z, N
0001	SUB	R = A - B = A + ~B + 1	[3:0]	1	C, V, Z, N
0010	AND	R = A & B	[3:0]	1	Z, N
0011	OR	R = A | B	[3:0]	1	Z, N
0100	XOR	R = A ⊕ B	[3:0]	1	Z, N
0101	NOT	R = ~A	[3:0]	1	Z, N
0110	SHL	R = A ≪ 1	[3:0]	1	C, Z, N
0111	SHR	R = A ≫ 1	[3:0]	1	C, Z, N
1000	MUL	R = A × B	[7:0]	4	Z
1001	DIV	R[3:0] = A/B R[7:4] =A%B	[7:0]	4	Z, div_by_zero

2. Full Adder Truth Table

A	B	Cin	Sum (S)	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

3. Single-Cycle Operations

Op	Code	A	B	Result	C	Z	V	N
ADD	0000	5	3	00001000 (8)	0	0	1	1
ADD	0000	15	1	00000000 (0)	1	1	0	0
SUB	0001	5	3	00000010 (2)	1	0	0	0
SUB	0001	3	5	00001110 (14)	0	0	0	1
AND	0010	12	10	00001000 (8)	0	0	0	1
OR	0011	12	10	00001110 (14)	0	0	0	1
XOR	0100	12	10	00000110 (6)	0	0	0	1
NOT	0101	12	0	00000011 (3)	0	0	0	0

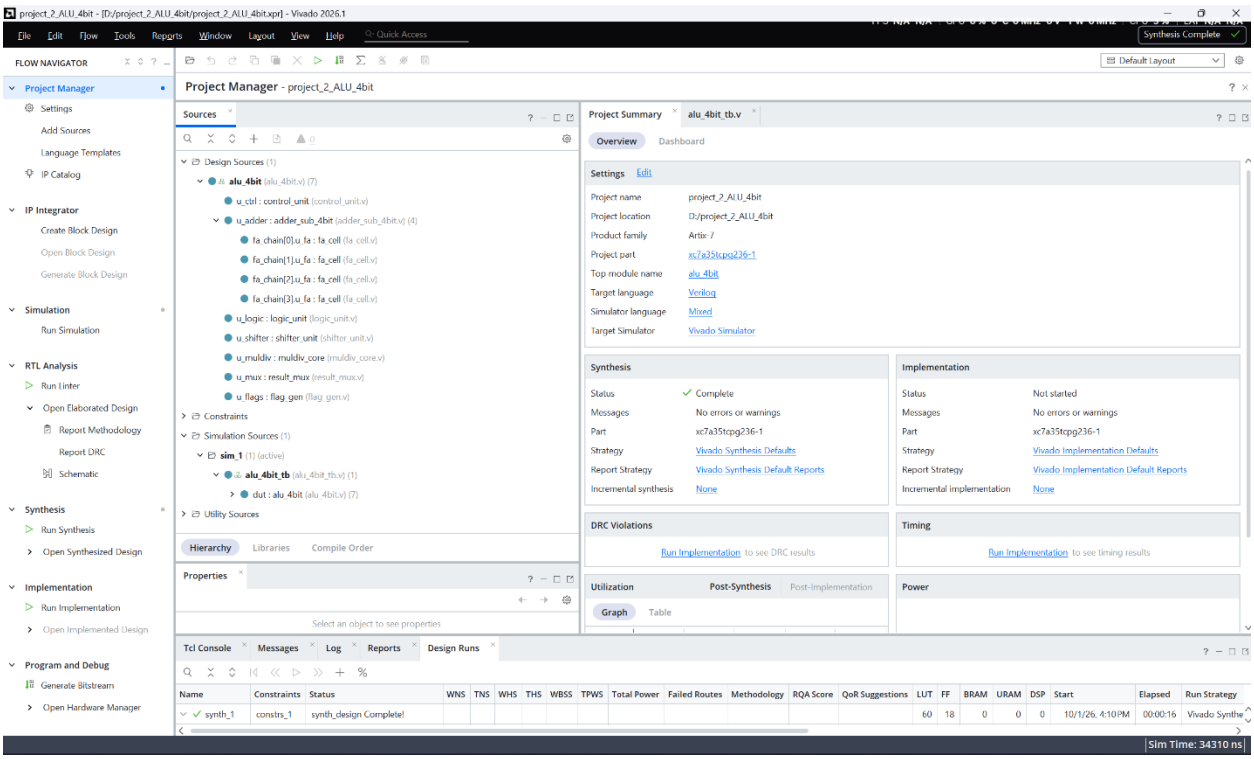
Op	Code	A	B	Result	C	Z	V	N
SHL	0110	11	0	00000110 (6)	1	0	0	0
SHR	0111	11	0	00000101 (5)	1	0	0	0

4. Multi-Cycle Operations

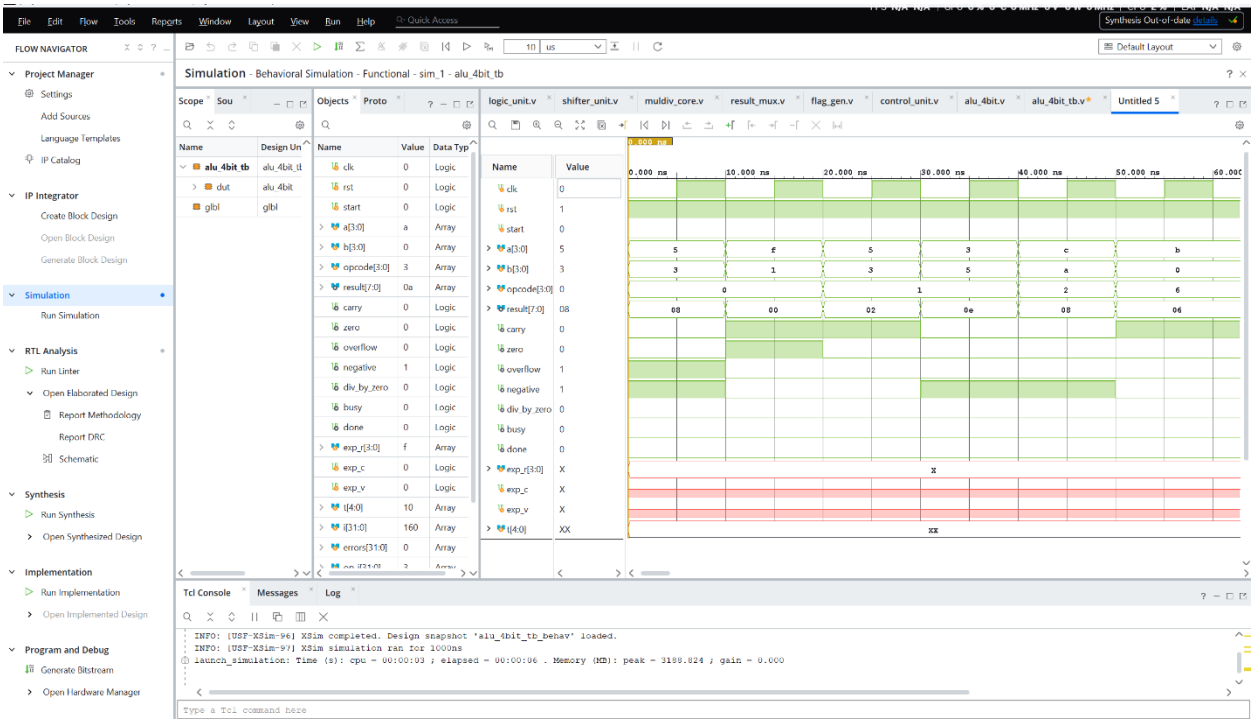
Op	Code	A	B	Result	Meaning	Cycles
MUL	1000	13	11	10001111	product = 143	4
MUL	1000	15	15	11100001	product = 225	4
DIV	1001	13	4	00010011	quot = 3, rem = 1, dbz = 0	4
DIV	1001	15	3	00000101	quot = 5, rem = 0, dbz = 0	4
DIV	1001	9	0	10011111	quot = 15, rem = 9, dbz = 1	4

Part D. Tool Flow Evidence & Screenshots

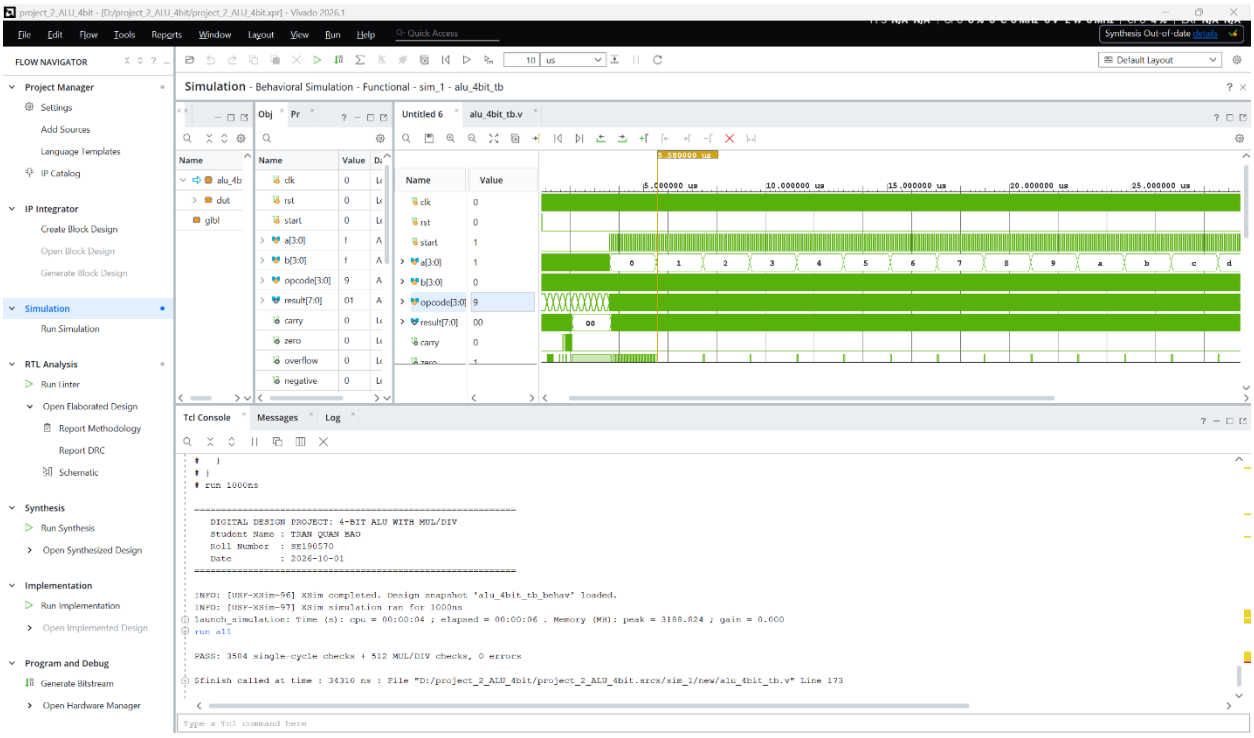
The six screenshots below are the required F1-F6 evidence. The project name is visible in the Vivado window



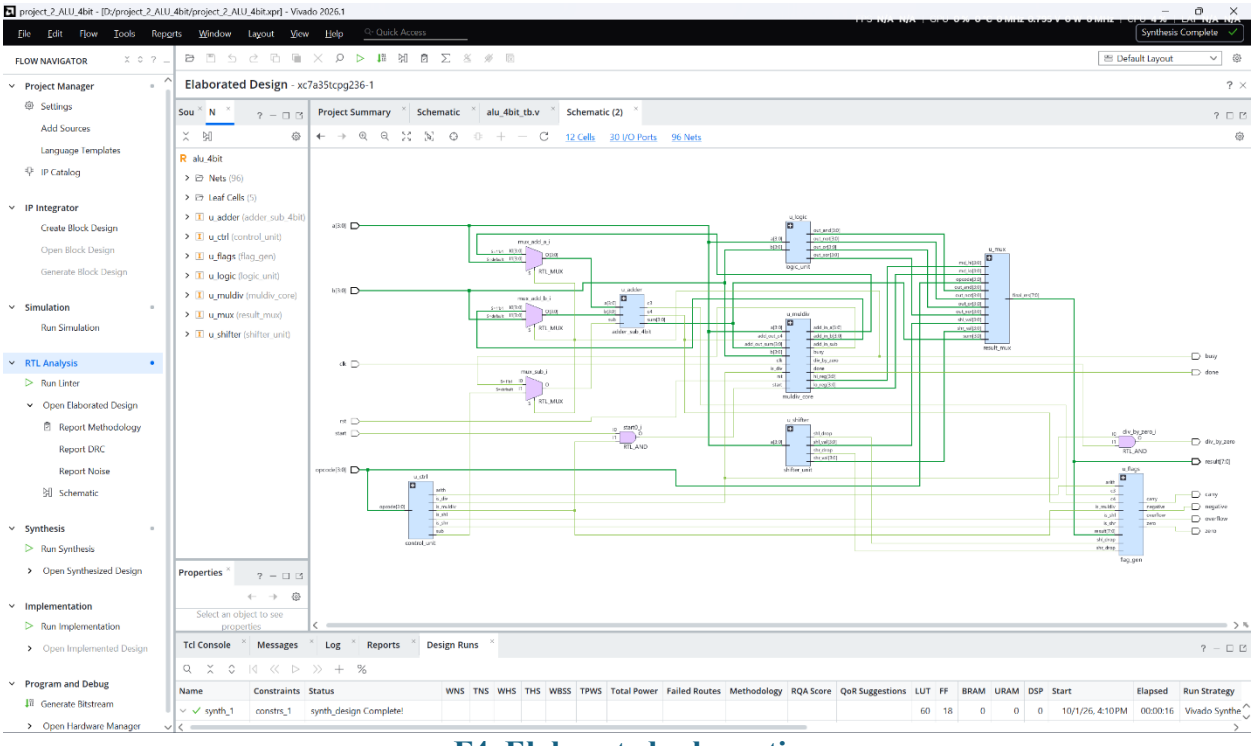
F1. Sources window - design hierarchy and simulation source



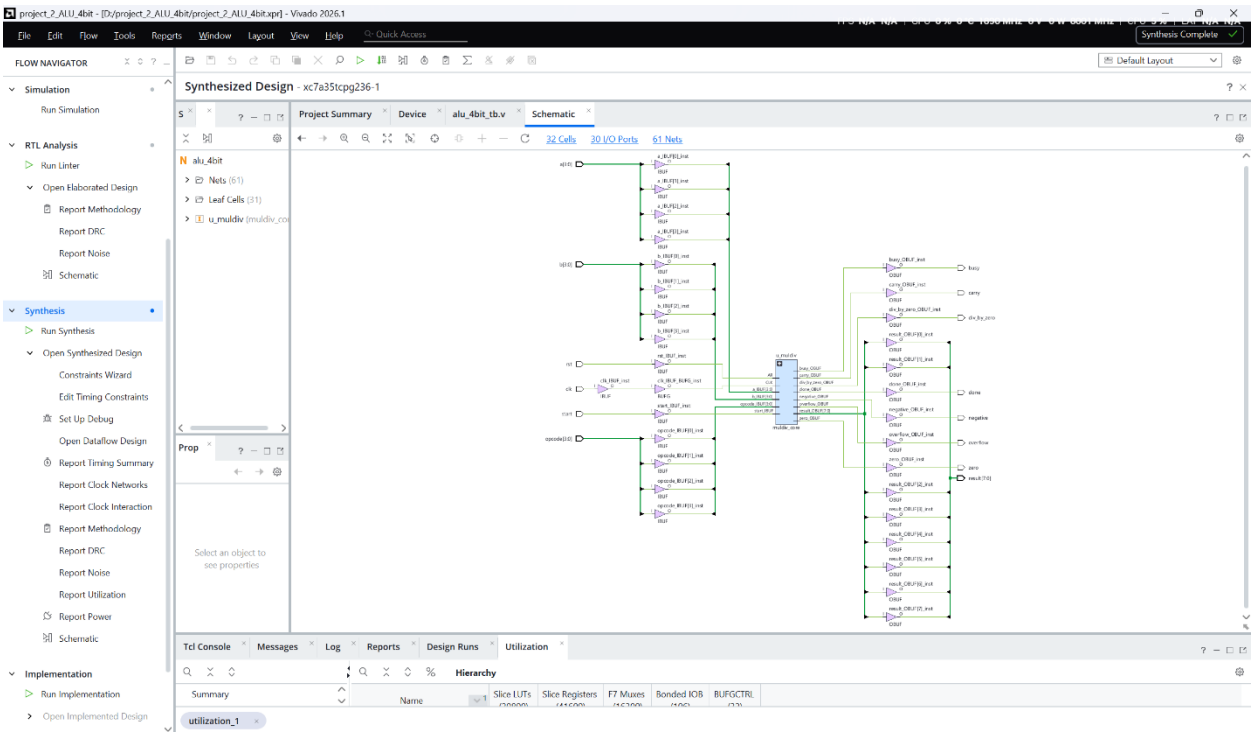
F2. Behavioral simulation waveform



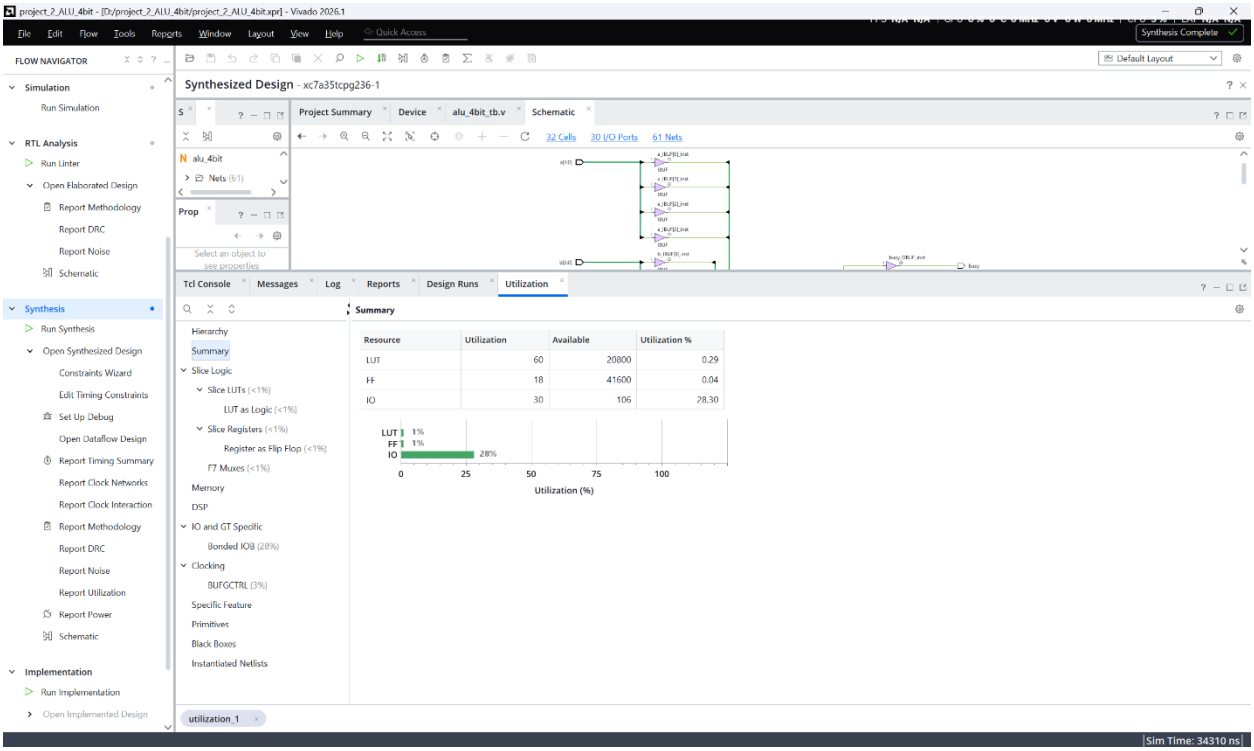
F3. Tcl Console - student ID and PASS result



F4. Elaborated schematic



F5. Synthesized schematic



F6. Utilization report

Part E. Flow Explanation

Step 1: Create Project & Select Component (xc7a35tcpg236-1)

Action: I initialized the project and selected the specific Artix-7 chip model (xc7a35tcpg236-1).

Key Reason: This ensures that Vivado optimizes logic and calculates latency based on the actual hardware limits and pin constraints of the FPGA being used.

Step 2: Add sources (Design vs. Simulation)

Action: I place all synthesizable modules into the "Design Sources" section and separate the `alu_4bit_tb.v` file into the "Simulation Sources" section.

Reason: Design sources translate into actual physical logic gates on the chip, whereas the testbench runs virtually within software memory

Step 3: Behavioral Simulation

Action: I ran the entire verification suite in Vivado's XSim simulator.

Reason: This confirms the logic is sound before deploying to hardware; all eight single-cycle operations and two multi-cycle operations passed every test with zero errors.

Step 4: RTL Analysis (Elaborated Design)

Action: I generated an elaborated schematic to examine how the Verilog modules are interconnected.

Reason: This step allows me to verify data flow and control connections across distinct, technology-independent functional blocks (such as adders, multiplexers, and registers) prior to the mapping stage.

Step 5: Synthesis (Synthesized Design)

Action: I synthesized the RTL logic into actual Xilinx hardware primitives.

Reason: Vivado translates my generic equations into real physical components: 6-input LUTs for logic, FDRE flip-flops for sequential states, and IBUF/OBUF for external pins.

Step 6: Utilization Report

Action: I inspected the resource utilization report to check the hardware area my ALU consumed.

Reason: It validates that sharing a single adder kept resource usage remarkably low:

- **LUTs:** 60 / 20,800 (0.29%)
- **Flip-Flops:** 18 / 41,600 (0.04%)
- **IO Pins:** 30 / 106 (28.30%)